SoK: The Widening Gap Between Adversarial Patch Attacks and Modern Object Detectors

Zhihe Zhang[1,a)] Shuo Peng[1,b)] Tatsuya Mori[1,2,3,c)]

Abstract: Adversarial patches are a well known physical threat to object detection, and many attacks report reducing a detector's detection rate to nearly zero. Surveying 77 patch attacks published from 2017 to 2026, however, we find that the detectors these attacks are evaluated on have not kept pace with the progress of object detection: the median gap between a paper and the detectors it attacks has widened to more than seven years, and even scoring each paper on the newest detector it evaluates leaves a gap of nearly four years. This leaves an open question: do the high attack success rates reported in prior work still hold on the newest and largest detectors? To answer it, we systematize published attacks into five stages by where each attack trains its patch, from raw pixels to text prompts, and evaluate representative attacks against 40 released detectors under a single white-box protocol, training 160 patches from scratch. Our evaluation yields two findings. First, run as published, most attacks lose much of their strength as the detector becomes newer and as the variant within a family grows larger, leaving mAP@0.5 at 75.7 on the extra-large modern variants against 28.9 on the darknet era. Second, the detectors the literature attacks most often are also the ones our evaluation finds easiest, so the choice of victim is not a neutral sampling decision, and a reported success rate describes the corner of the detector space the field has been working in, not the space as a whole. We hope our survey, protocol, and measurements help the community evaluate patch attacks against up-to-date detectors.

Keywords: adversarial patch, object detection, systematization of knowledge, threat assessment, evaluation methodology

1. Introduction

Object detectors now sit inside perception pipelines that act on what they report. One tells a vehicle that a pedestrian is ahead, while another tells a monitoring system that a person is in the frame. An input that erases a person from the output is therefore a physical-world threat, and its canonical form is the adversarial patch, a printed or wearable graphic optimized against a detector so that the detector stops reporting the person who carries it [5], [12], [41]. Papers in this line routinely report the detection rate falling to almost nothing [17], [18], [41], [48], and a more recent branch makes the patch look like an ordinary object so that the attack hides inside a plausible image [17], [32], [47].

While these attacks accumulated, object detectors themselves kept evolving. The YOLO line alone went from the darknet-era v2 through v4 [3], [36], [37], through the ultralytics rewrite [22], through the anchor-free redesign of v8 [23], and on to v9, v10, and v11 [24], [42], [45]. Every step reworked the backbone, the head, or the training recipe, and every step localized people more reliably than the one before. The attacks did not follow. We surveyed 77 patch attacks published from 2017 to 2026 and recorded each detector that every one of them targets. Faster R-CNN [38] and YOLOv3 [37] remain the two most common victims; the newest generations are almost never chosen, and the lag is not shrinking. As **Fig. 1** shows, the rows for the recent detectors stay nearly empty while the frontier climbs past them, and the median distance between a paper and the detectors it attacks ranges from 0.8 years for the 2017 papers to 7.7 years for those appearing in 2026. It is understandable that each attack was evaluated on the detectors that were available or common at the time. However, results measured on superseded detectors say little about the detectors deployed today, and this leaves an open question: *Do the high attack success rates reported in prior work still hold on the newest and largest detectors?*

To answer this question, we present a systematization of knowledge (SoK) on adversarial patch attacks and their victim detectors. We first survey how each side evolved, the object detection side and the adversarial patch attack side, and by comparing their design updates, we establish that a gap between the victim models and the attacks does exist. To clarify what that gap costs, we then carry out a replication study. We select four representative attacks and run each of them against 40 released detectors under one white-box matched protocol, training every patch from scratch on

[1] Waseda University
[2] RIKEN AIP
[3] NICT
[a)] zzhihe@nsl.cs.waseda.ac.jp
[b)] hosaku@nsl.cs.waseda.ac.jp
[c)] mori@nsl.cs.waseda.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

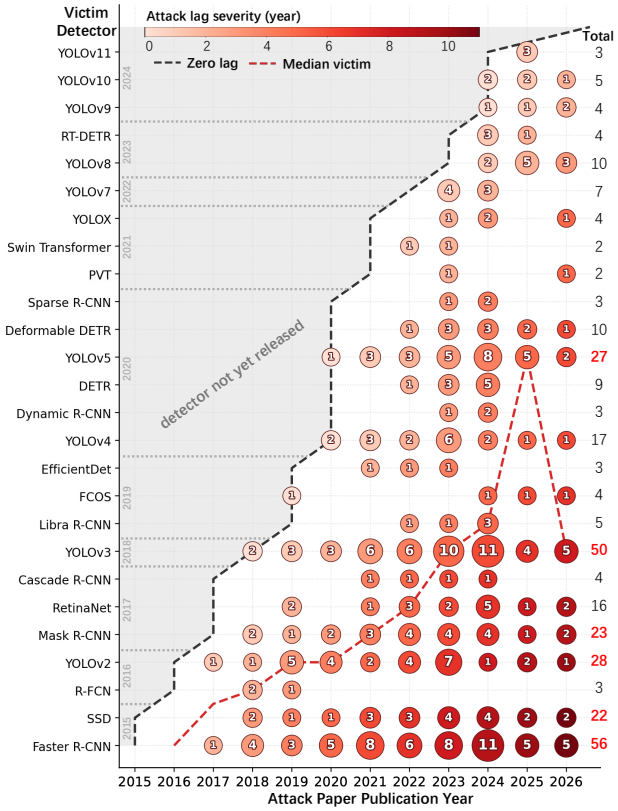


Fig. 1: Victim detectors of the 77 surveyed attacks, one row per detector family with size variants merged. A bubble counts that year's papers attacking that family and is colored by the lag between the two, and the right column totals the papers per family. Zero lag is the newest detector available when a paper appeared, so the gray region is a detector not yet released. The median victim never leaves the 2015 to 2020 band while that frontier climbs to 2024.

the detector it will face.

The replication study shows that this gap indeed matters. Each attack is strong against the darknet-era detectors and the smallest modern variants, and each one weakens as the detector becomes newer and as the variant within a family grows larger, until on the extra-large modern models none of them comes close to working. Worse, the detectors that the field picks most often are also the ones our evaluation finds easiest, so the choice of victim is not a neutral sampling. The two findings together suggest that a reported success rate describes the corner of the detector space the field has been working in rather than the space as a whole.

In summary, this paper makes the following contributions.

- We survey 77 patch attack papers published from 2017 to 2026 and record every victim detector they report. The survey quantifies the gap between the detectors these papers attack and the detectors available at the time.
- We evaluate four representative attacks against 40 released detectors under a single white-box matched protocol, training 160 patches from scratch. The evaluation shows that attack effectiveness drops sharply on newer and larger detectors, so the high success rates reported in prior work do not carry over to them.
- We show that the most frequently attacked detectors are also the ones our evaluation finds most vulnerable. Published success rates were thus measured on the easiest targets and overestimate the threat to modern detectors.

Section 2 reviews how detectors and patch attacks each evolved, Section 3 measures the gap between them, and Section 4 shows what the gap costs. Section 5 states the limits of the study and what follows for the field.

2. Background

2.1 The Evolution of Object Detectors

Object detection asks a model to find every instance of a class, box it, and label it. Two-stage detectors such as R-CNN and Faster R-CNN [15], [38] propose regions first and classify them second. One-stage detectors such as SSD [30], RetinaNet [29], and YOLO [35] drop the proposal step and predict boxes densely in one pass. Transformer detectors such as DETR [6] later recast detection as set prediction and removed anchors and non-maximum suppression. All of them end at the same gate, where every candidate carries a confidence score and only those above a threshold reach the output.

Across these generations, the main change lies in how this confidence gate is constructed. The gate is exactly what a patch attack must defeat, so its redesign directly affects how the attack works. In the darknet generations of YOLOv2 to v4 [3], [36], [37], each anchor predicts an explicit objectness score, trained with its own loss and multiplied by the class probability to form the box's confidence. That hands the attacker one clean lever, since forcing objectness toward zero collapses the person's confidence no matter how clearly the region still looks like a person. YOLOv5 [22], YOLOv7 [43], and YOLOR [44] keep this formulation. The modern generations take the lever away. YOLOv6 [27] drops the objectness branch and YOLOX [14] drops the anchors, and the anchor-free YOLOv8 [23] folds confidence directly into the per-class score, so there is no standalone number left to push down. YOLOv9 [45] adds programmable gradient information, YOLOv10 [42] removes non-maximum suppression, and YOLOv11 [24] refines the backbone again.

Besides the generation of the architecture, detectors also differ along a second axis: model size. Each generation ships as a family of sizes, and the large variants localize people far more reliably than the small ones, so the strongest detector available at any moment is both recent and large. An attack inherited from the darknet era therefore faces a target that has moved on both counts at once.

2.2 The Evolution of Patch Attacks

The adversarial patch was introduced by Brown et al. [5], who showed that a single carefully optimized image placed in a scene can manipulate the output of a deep learning model. The goal of the attacks that target object detection is evasion: the patch is optimized so that the confi-

© 2026 Information Processing Society of Japan

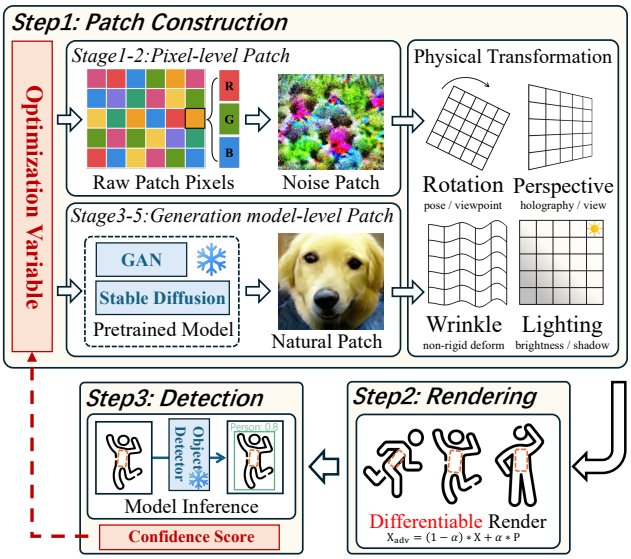


Fig. 2: The training pipeline all patch attacks share. Only the optimization variable differs between the two branches, the patch pixels for Stages 1 and 2 and the generation model's input for Stages 3 to 5. Everything downstream of it is common to all five.

dence the detector reports on the person carrying it falls under the gate of Section 2.1, and that person is dropped from the output. Attackers reach it through the training strategy drawn in **Fig. 2**, which constructs the patch, applies physical transformations such as rotation and lighting, composites it onto the person with a differentiable renderer, and reads the frozen detector's confidence on that person as the loss, whose gradient returns to whatever variable the method exposes. The methods differ only in which variable they optimize, and that choice divides the literature into five stages.

Stage 1 covers the earliest attacks, which optimize the patch's own pixel values. Thys et al. [41] first carried the patch onto an object detector and opened the line of work that hides a person with one, and the attacks of this period [2], [19], [33] take the most direct route available, adjusting pixel values by backpropagation to drive the confidence score down. **Stage 2** folds in physical conditions. Attackers began to inject lighting and cloth wrinkle into training to narrow the gap between a digital attack and a physical one, and the methods gathered in [16], [18], [20], [31], [40], [46], [48], [49] are classic designs of this kind.

Neither stage asks the result to resemble anything, so what comes out reads as noise and is noticed long before it fools a detector. To turn the patch into a natural patch, researchers began to build it with a generation model. **Stage 3** works through a GAN [10], [17], [34], [51], the generation model available when the line began, and **Stage 4** follows once diffusion models displace GANs, steering the patch during denoising instead [8], [28], [32], [47]. Both of those optimize a latent code, whereas **Stage 5** moves upstream of the code and optimizes the text prompt handed to a text-to-image model [21], [25], [26]. Realism improves down the sequence, and so does the constraint, since a patch built this way can only ever be an image the frozen model can draw.

3. Victim Detector Selection

3.1 Distribution of Victim Detectors

Our survey covers 77 papers published from 2017 to 2026. We include a paper when it trains a patch against an object detector, whether the attack is digital or physical and whatever the target class, and we exclude patches aimed at classifiers rather than detectors along with attacks on face recognition, re-identification, and segmentation. Dates come from the arXiv v1 of the paper and of the victim detector. For each paper we recorded every detector it reports attacking, together with the variant where one is named.

Fig. 1 is the result. By age, the field still overwhelmingly targets detectors from 2015 to 2020. Faster R-CNN and YOLOv3 alone are attacked by 56 and 50 of the 77 papers. YOLOv8, available since 2023, is attacked by 10, and YOLOv9, YOLOv10, and YOLOv11 by 4, 5, and 3. The rows for the newest detectors stay nearly empty throughout. The size axis tells the same story less visibly, because papers often omit the variant altogether. Among the 65 YOLO mentions that do name one, the nano, tiny, and small variants account for 46. Only two papers in the entire survey attack a large or extra-large YOLO variant, one on YOLOv8x and one on YOLOv10b. **Table A.1** places 24 methods in the five stages of Section 2.2, and Faster R-CNN appears in 14 of them, which makes a detector from 2015 the most common victim in a list that runs to 2025.

3.2 Temporal Lag Analysis

Naming an old detector is not by itself a fault, since a 2019 paper could hardly attack a 2023 model. What matters is the distance measured at the time each paper was written. Taking each paper's publication year against the release year of every detector it attacks, the median lag was 0.8 years for the 2017 papers, which is to say those papers attacked what was then current. It reaches 3.0 years by 2020 and stands at 7.7 years for the papers appearing in 2026.

A reader might suspect this is an artifact of how papers assemble their victims, since an attack on a modern detector often keeps an older one alongside it for comparison and that older victim drags the average back. **Fig. 3** removes the effect by scoring each paper on the newest detector it evaluates, which is the smallest lag that paper could report. The lag survives. Measured that way it still reaches 3.9 years in 2026 against 1.5 years in 2018, and it rises alongside the mean rather than staying flat. The field is choosing victims that were already several years old when the work appeared, not merely keeping an older baseline alongside a modern one.

One recent study has already pushed in this direction. It adapts a naturalistic attack to several modern YOLO detectors, finds that the larger ones resist, and attributes the

© 2026 Information Processing Society of Japan

Table 1: The evaluation over 40 released detectors, mAP@0.5 above and ASR below. Both metrics weaken as the detector gets newer and as the variant grows larger.

Attack	v2	v3	v3t	v4	v4t	v5n	v5s	v5m	v5l	v5x	v6n	v6s	v6m	v6l	v7t	v7	v7x	v8n	v8s	FRCNN
mAP@0.5 ↓																				
AdvPatch [41]	26.2	35.1	16.3	47.9	29.1	27.9	25.5	50.0	55.2	71.2	26.2	24.2	57.5	37.3	25.1	44.9	54.0	41.8	35.6	30.9
AdvTexture [18]	23.6	70.2	40.2	10.6	22.8	34.4	22.2	46.4	60.5	71.0	36.3	29.3	44.7	48.2	21.7	49.5	57.0	28.3	26.2	24.8
NAP [17]	28.2	56.2	8.5	13.5	21.5	40.6	53.1	65.7	74.4	79.7	75.4	69.5	75.8	66.4	67.1	60.9	65.5	51.1	69.5	67.8
BadPatch [47]	17.9	21.6	14.7	62.5	12.4	20.6	35.2	61.7	76.0	79.7	25.3	30.9	32.6	47.0	14.5	40.4	50.1	30.1	37.1	31.3
ASR ↑																				
AdvPatch [41]	60	49	76	58	76	73	70	60	37	33	64	69	38	44	74	55	40	70	63	45
AdvTexture [18]	58	26	55	88	80	70	78	59	41	30	60	64	54	41	73	42	33	75	70	57
NAP [17]	55	39	79	83	73	60	44	44	35	31	29	31	30	31	35	43	37	60	40	23
BadPatch [47]	68	62	84	44	86	83	74	52	38	33	62	64	61	47	80	64	52	76	69	43
Attack	v8m	v8l	v8x	v9t	v9s	v9m	v9c	v9e	v10n	v10s	v10m	v10b	v10l	v10x	v11n	v11s	v11m	v11l	v11x	RTDETR
mAP@0.5 ↓																				
AdvPatch [41]	52.1	66.2	76.7	60.1	62.6	58.4	61.2	81.2	62.0	41.5	60.4	52.5	81.2	66.3	71.3	59.3	66.0	70.0	65.7	43.0
AdvTexture [18]	34.2	78.2	83.5	77.9	75.0	57.8	79.6	86.2	33.9	23.6	69.0	57.3	79.2	84.9	30.9	21.4	38.7	72.5	77.5	61.4
NAP [17]	68.6	76.2	75.5	70.2	68.2	62.7	82.9	76.1	46.6	58.8	68.1	67.8	81.6	77.3	63.3	51.0	51.7	79.2	73.4	65.5
BadPatch [47]	56.9	75.6	74.9	22.3	34.0	59.4	74.5	77.5	22.4	17.0	31.3	58.0	50.4	63.1	42.2	32.4	37.9	66.5	73.1	18.7
ASR ↑																				
AdvPatch [41]	52	41	33	51	48	48	39	29	56	65	50	53	28	39	38	48	42	39	41	50
AdvTexture [18]	62	30	26	36	36	48	30	22	72	76	40	48	23	26	67	74	66	36	28	33
NAP [17]	42	30	27	43	41	47	28	30	63	50	41	44	29	35	43	51	46	32	36	33
BadPatch [47]	55	40	36	79	64	54	40	36	84	82	72	55	62	51	66	71	74	51	41	72

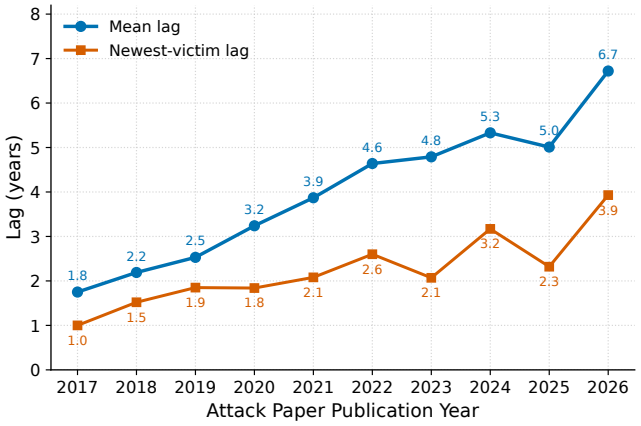


Fig. 3: Lag between a paper and the detectors it attacks. The mean lag averages over every detector a paper evaluates, while the newest-victim lag counts only the newest one and is therefore the smallest lag that paper could report. Both rise across the decade.

resistance to their greater capacity [13]. That is the closest prior attempt to ask our question, and it concludes that modern and large detectors largely withstand patch attacks. Our evaluation revisits the conclusion with a wider attack axis and arrives at a different explanation for the same observation.

4. Cross-Detector Evaluation of Patch Attacks

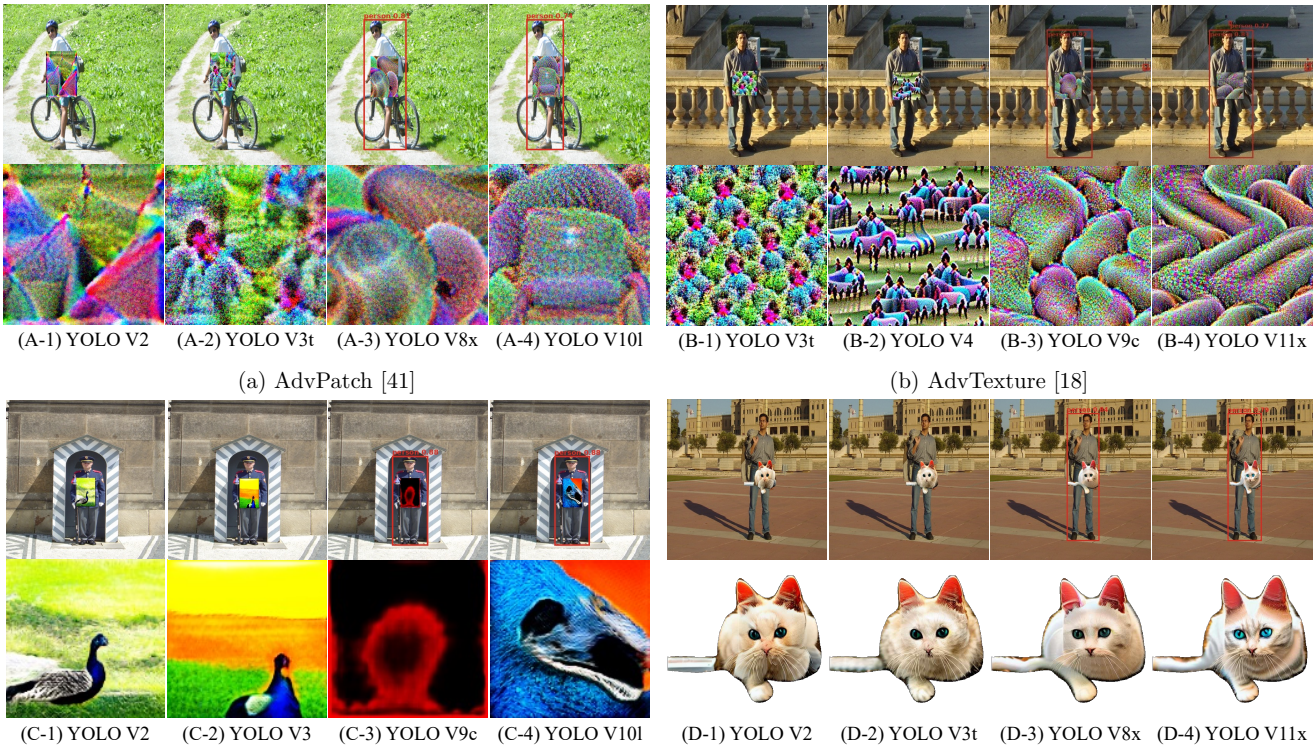
4.1 Evaluation Protocol

We model the adversary as the literature describes: white-box and matched, with full access to the victim's architecture and weights, optimizing the patch on that detector and scoring it on the same one, so neither a train-test mismatch nor a transfer penalty affects the numbers. Everything is digital. Each patch is composited onto the image, and none of the printing, camera, and lighting losses a worn patch incurs are charged against it. Every one of those choices favors the attacker, so the bias of the setup runs against our own conclusion rather than toward it.

We select one representative per stage based on two criteria. It must ship an official open-source implementation for person evasion, and among those, it must instantiate the stage's defining search space in the plainest form, so that one method stands in for its stage without dragging in machinery particular to one paper. This yields AdvPatch [41] for the pixel stage, AdvTexture [18] for the physical stage, NAP [17], which optimizes the latent of a frozen BigGAN [4] for the GAN stage, and BadPatch [47], which optimizes a latent of Stable Diffusion [39] for the diffusion stage. Stage 5 offers no candidate since EMA and PG-ECAP release no code, and AngleRoCL releases code that attacks stop signs rather than people, so that stage stays in the taxonomy and out of the evaluation. Table A.1 records the code we found for all 24 methods.

For victims, we take the YOLO family from v2 to v11 together with Faster R-CNN and RT-DETR [50], 40 released detectors in all. YOLO is the only line published both as a series of generations and as a family of sizes within each, which lets the release year and capacity vary separately inside one codebase. Faster R-CNN is the most attacked victim in our survey, and RT-DETR is a transformer that predicts a set one-to-one and runs no non-maximum suppression. Each attack runs from its authors' code, nothing is retuned when it moves to a new detector, and the only thing we add is the interface to an unsupported detector, which reads per-box person confidence as objectness times class probability on darknet-style heads and as the bare class score on anchor-free ones. Appendix A.4 gives the rest of the configuration. Every patch is trained and evaluated on the same detector, so each cell of **Table 1** is one attack against one detector, 160 patches in all. Each cell is a single training run at one fixed random seed, with the conditioning class of the two generative attacks held fixed as well, so no number averages over configurations an attacker would have to

© 2026 Information Processing Society of Japan



(a) AdvPatch [41] (b) AdvTexture [18]

(c) NAP [17] (d) BadPatch [47]

Fig. 4: Trained patches and the detector's output for each evaluated attack. Within a panel the upper image is the detection on the composited scene, where a box means the person is still found, and the lower image is the patch itself. The two left columns of every panel are darknet-era victims and the two right are modern ones, so the same attack empties the left pair and leaves the right pair standing.

choose between. Appendix A.4 names the settings. Experiments use the INRIA Person set [9], with ground truth taken to be each detector's own clean-image detections, which fixes the clean mAP at exactly 100% and pins any drop on the patch. We report *mAP@0.5* at IoU 0.5 with VOC interpolation [11], where lower looks stronger, and *ASR*, the fraction of persons no longer reported at a confidence of 0.5, where higher looks stronger.

4.2 Evaluation Results

Table 1 gives the result, and **Fig. 4** shows what the trained patches do. We read the table along the two axes that the YOLO line separates, the release year and the size of the variant, and **Fig. 5** collapses it onto both.

Take the generation first. Every attack empties the darknet era, where the four of them leave a mean mAP of 28.9, and every attack loses ground as the head is redesigned around it. The anchor-free and NMS-free generations cost the most, with the four attacks leaving 76.7, 83.5, 75.5, and 74.9 on YOLOv8x, and the hardest cells of the whole table appearing on YOLOv9e at 86.2 and YOLOv10x at 84.9. The decline is not uniform within a generation, though. BadPatch still reaches 17.0 on YOLOv10s and 18.7 on RT-DETR, and AdvTexture reaches 21.4 on YOLOv11s, so a recent head is not categorically out of reach. The generation axis raises what the hardest members of a family cost, and RT-DETR sits where that reading predicts, at a mean mAP of 47.2 and a mean ASR of 47.

The size axis moves the same way and moves further. Within one generation, the codebase, the recipe, and the released weights come from one lineage, and only the width of the network changes; yet, that alone spans most of the range the table covers. On YOLOv5, BadPatch runs from 20.6 on v5n to 79.7 on v5x, and AdvPatch from 27.9 to 71.2. On YOLOv8, AdvTexture runs from 26.2 on v8s to 83.5 on v8x. Averaged over the four attacks, the smallest modern variants are left at 43.4 against 75.7 for the extra-large ones, so one unreported letter in a model name is worth more than thirty mAP points, against 18.6 between the four attack designs themselves. Section 3.1 found two papers in the whole survey that attack a large or extra-large variant.

Fig. 5 puts both readings in one place. The two left panels plot mAP and the two right panels ASR, each of them against capacity and then against head generation. All eight curves run the same way. The mAP climbs as the detector grows larger or newer, the ASR drops across the same span, and both movements mean the same thing: that less of the person is being hidden. The two measures are computed independently of each other, so the weakening sits in the attacks and not in how we score them. The table indicates how far it goes and where it leaves the field. Across the five darknet detectors, the strongest and the weakest of the four attacks stand 31.8 mAP points apart on average, and across the five extra-large modern variants, they are only

© 2026 Information Processing Society of Japan

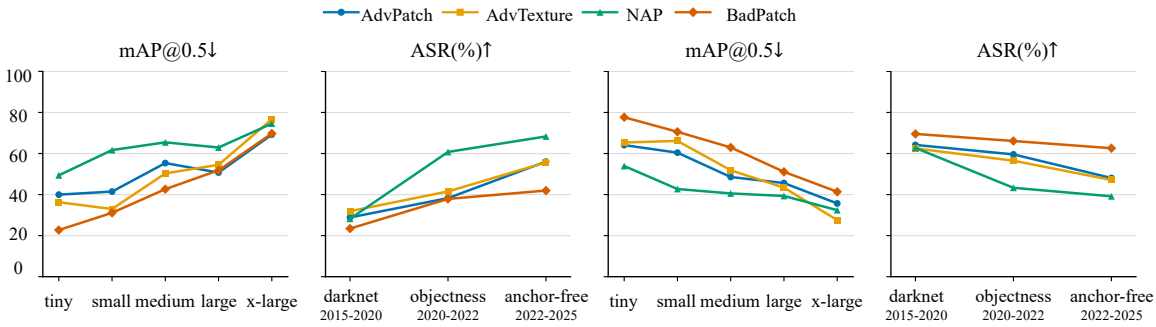


Fig. 5: Attack strength averaged over the detectors of each capacity tier and of each head generation, under both metrics. Tiers follow the released variant name and the two non-YOLO detectors are placed by parameter count. Every attack loses strength along both axes, and the two metrics agree on the direction while disagreeing on how strong any single attack is.

12.2 apart, with all four above 65. At the easy end of the range, the design of the attack decides a great deal. At the hard end, it decides little besides the choice of victim, so what a decade of work in this literature added is least visible exactly where a threat estimate would need it.

Ranking all 40 detectors by the mean mAP that the four attacks leave on them puts that decline alongside the survey of Section 3. The six detectors that ten or more papers target all fall within the easiest sixteen, four of the six within the easiest six, and those popular victims average 30.6 while the thirty-four the field passes over average 55.0. Faster R-CNN is the sharpest case, since the single most attacked detector in the survey is also one that three of the four attacks leave between 24.8 and 31.3, roughly what they achieve in the darknet era.

None of this requires anyone to have chosen an easy target on purpose. A detector becomes a common victim because code exists for it and because earlier papers used it. Those same properties make it the target that the attack pipelines were already shaped around, so convenience and weakness arrive together. Either way, a patch attack is offered as a general capability against person detection rather than as a result about one particular network. Thus, a success rate collected at the easiest end of the range measures that end and not the capability.

5. Discussion

5.1 Limitations

The evaluation was built to treat every method as favorably as we could, and that design fixes what the numbers can be asked to support. Five boundaries are worth stating.

The protocol is digital on purpose. Compositing each patch onto the image and charging it none of the physical degradation a worn patch pays removes the confounds that could otherwise be offered as explanations for the decline we measure, and what remains is an upper bound on physical strength. The same setting withholds from Stage 2 the robustness machinery it was built for, since a method that trains against cloth deformation is never asked here to survive a moving garment. A physical trial could therefore reorder the methods that spend capacity on real-world robustness.

The attack axis is thinner than the detector axis. Forty detectors give the victim side close to full coverage, while each stage stands on a single representative chosen for being the plainest instance of its kind, as Table A.1 sets out. Our numbers hold for these four methods on these forty detectors, and reading them as claims about a whole stage rests on the representative speaking for its stage. No method in Stage 5 releases code we could run on person detection, so we discuss that stage qualitatively and do not measure it.

Each cell is one training run, so the table carries no estimate of how far a cell would move under a different draw. Putting error bars on the decline we report would mean repeating all 160 of them, and we leave that undone.

Every number comes from INRIA Person, the benchmark this literature has standardized on. Its images are low in resolution and its scenes narrower than what a deployed detector meets today, so a larger and more recent dataset would test whether the trend we report holds at higher resolution. The dataset cannot explain away the ordering, since every attack meets every detector on the same images.

Finally, we report the two metrics the field reports, and neither measures hiding cleanly. The mAP rewards precision, so a patch can drag it down by spawning decoy boxes without hiding anyone. Both metrics also credit a patch for the occlusion that any opaque block of the same footprint would cause. Absolute values should therefore be read as an upper bound on hiding, though the ordering is unaffected, since the same inflation applies to every attack on every detector.

5.2 Implications

The defense side of this literature settled an equivalent question years ago. Carlini et al. drew the lesson that a defense is credible only against the strongest attack available rather than the one most convenient to run [1], [7], and a defense paper today is expected to face the best method its authors can find. The attack side has never adopted the mirror image of that rule, although the logic is identical. When an attack is measured on a detector the field has already moved past, the reported success rate describes the

© 2026 Information Processing Society of Japan

target rather than the attack.

Our recommendation is a reporting standard, not a new technique. A method offered as a real threat should show a significant effect against the strongest and most recent detectors available when it is published, and should say which variant it used, since our evaluation finds the size axis alone worth tens of mAP points. Beating an older baseline establishes progress with respect to that baseline and does not establish a threat. One reply available to the field is that the older detectors are the ones actually deployed, so attacking them is the realistic choice. Deployment moves as well, since edge devices grow more capable and the models they run grow stronger with them, so an already obsolete victim only postpones the question.

What our measurements do not settle is why the decline happens, and the answer matters for what a defender should conclude from it. A modern detector may resist because it localizes people more reliably, or the attacks may simply be aimed at design choices the modern heads no longer make, and the two readings call for different responses. Separating them needs an attack side rebuilt against current heads rather than a wider sweep of detectors, and we leave it open.

None of this is evidence that patch attacks have stopped working. On the detectors the field actually reports, they work about as well as the papers say, at a mean mAP of 28.9 across the darknet era. The gap shows instead that those numbers were collected where the attack is easy, so they cannot be read as an estimate of present risk in either direction.

6. Conclusion

In this paper, we asked which detectors the adversarial patch literature actually attacks, and whether the high success rates it reports still hold on the detectors that exist. Surveying 77 attacks published from 2017 to 2026, we found the median distance between a paper and its victims running from 0.8 years in 2017 to 7.7 years in 2026, with Faster R-CNN and YOLOv3 still being the two most common targets and only two papers attacking a large YOLO variant at all. Evaluating four representative attacks against 40 released detectors under one matched protocol, we found that the reported threat does not survive that distance. The four attacks leave the darknet-era models at a mean mAP of 28.9 and the extra-large modern ones at 75.7, and the detectors the field targets most are the ones our evaluation finds easiest. An attack earns the name of a threat when it holds for the newest and largest models as well as for the old and small ones, and we raise the point because defenders spend their effort where the field's threat estimates point.

Acknowledgments This work was partially supported by JST CREST JPMJCR23M4 and by JST BOOST JPMJBS2429.

References

[1] Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 274–283. PMLR, 2018.

[2] Jens Bayer, Stefan Becker, David Münch, and Michael Arens. Eigenpatches: Adversarial patches from principal components. In *Advances in Visual Computing*, 2023.

[3] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[4] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *International Conference on Learning Representations (ICLR)*, 2019.

[5] Tom B. Brown, Dandelion Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. Adversarial patch. In *NIPS Workshop on Machine Learning and Computer Security*, 2017. arXiv:1712.09665.

[6] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 213–229, 2020.

[7] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*, 2019.

[8] Xianyi Chen, Fazhan Liu, Dong Jiang, and Kai Yan. Natural adversarial patch generation method based on latent diffusion model. *arXiv preprint arXiv:2312.16401*, 2023.

[9] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893. IEEE, 2005.

[10] Bao Gia Doan, Minhui Xue, Shiqing Ma, Ehsan Abbasnejad, and Damith C. Ranasinghe. TnT attacks! universal naturalistic adversarial patches against deep neural network systems. *IEEE Transactions on Information Forensics and Security*, 17:3816–3830, 2022.

[11] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[12] Kevin Eykholt, Ivan Evtimov, Earlence Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. Robust physical-world attacks on deep learning visual classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1625–1634, 2018.

[13] D. L. Gala, J. Molleda, and R. Usamentiaga. Evaluating the impact of adversarial patch attacks on YOLO models and the implications for edge AI security. *International Journal of Information Security*, 24(154), 2025.

[14] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding YOLO series in 2021. *arXiv preprint arXiv:2107.08430*, 2021.

[15] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 580–587, 2014.

[16] Amira Guesmi, Ruitian Ding, Muhammad Abdullah Hanif, Ihsen Alouani, and Muhammad Shafique. DAP: A dynamic adversarial patch for evading person detectors. *arXiv preprint arXiv:2305.11618*, 2023.

[17] Yu-Chih-Tuan Hu, Bo-Han Kung, Daniel Stanley Tan, Jun-Cheng Chen, Kai-Lung Hua, and Wen-Huang Cheng. Naturalistic physical adversarial patch for object detectors. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7848–7857, 2021.

[18] Zhanhao Hu, Siyuan Huang, Xiaopei Zhu, Fuchun Sun, Bo Zhang, and Xiaolin Hu. Adversarial texture for fooling person detectors in the physical world. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13307–13316, 2022.

[19] Hao Huang, Yongtao Wang, Zhaoyu Chen, Zhi Tang, Wenqiang Zhang, and Kai-Kuang Ma. RPATTACK: Refined patch attack on general object detectors. In *IEEE International Conference on Multimedia and Expo (ICME)*, 2021.

[20] Yuanhao Huang, Yilong Ren, Jinlei Wang, Lujia Huo, Xuesong Bai, Jinchuan Zhang, and Haiyang Yu. AdvReal:

© 2026 Information Processing Society of Japan

Physical adversarial patch generation framework for security evaluation of object detection systems. *arXiv preprint arXiv:2505.16402*, 2025.

[21] Wenjun Ji, Yuxiang Fu, Luyang Ying, Deng-Ping Fan, Yuyi Wang, Ming-Ming Cheng, Ivor Tsang, and Qing Guo. AngleRoCL: Angle-robust concept learning for physically view-invariant adversarial patches. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[22] Glenn Jocher. Ultralytics yolov5. `https://github.com/ultralytics/yolov5`, 2020.

[23] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8. `https://github.com/ultralytics/ultralytics`, 2023.

[24] Rahima Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024.

[25] Dehong Kong, Siyuan Liang, and Wenqi Ren. Environmental matching attack against unmanned aerial vehicles object detection. *arXiv preprint arXiv:2405.07595*, 2024.

[26] Chaoqun Li, Huanqian Yan, Lifeng Zhou, Tairan Chen, Zhuodong Liu, and Hang Su. Prompt-guided environmentally consistent adversarial patch. *arXiv preprint arXiv:2411.10498*, 2024.

[27] Chuyi Li, Lulu Li, Hongliang Jiang, Kaiheng Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, et al. Yolov6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.

[28] Shuo-Yen Lin, Ernie Chu, Che-Hsien Lin, Jun-Cheng Chen, and Jia-Ching Wang. Diffusion to confusion: Naturalistic adversarial patch generation based on diffusion model for object detector. *arXiv preprint arXiv:2307.08076*, 2023.

[29] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.

[30] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 21–37, 2016.

[31] Arman Maesumi, Mingkang Zhu, Yi Wang, Tianlong Chen, Zhangyang Wang, and Chandrajit Bajaj. Learning transferable 3d adversarial cloaks for deep trained detectors. *arXiv preprint arXiv:2104.11101*, 2021.

[32] Boming Miao, Chunxiao Li, Yao Zhu, Weixiang Sun, Zizhe Wang, Xiaoyi Wang, and Chuanlong Xie. Advlogo: Adversarial patch attack against object detectors based on diffusion models. *arXiv preprint arXiv:2409.07002*, 2024.

[33] Yisroel Mirsky. IPatch: A remote adversarial patch. *arXiv preprint arXiv:2105.00113*, 2021.

[34] Svetlana Pavlitskaya, Bianca-Marina Codău, and J. Marius Zöllner. Feasibility of inconspicuous GAN-generated adversarial patches against object detection. *arXiv preprint arXiv:2207.07347*, 2022.

[35] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[36] Joseph Redmon and Ali Farhadi. Yolo9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7263–7271, 2017.

[37] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[38] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[39] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022.

[40] Wenyi Tan, Yang Li, Chenxing Zhao, Zhunga Liu, and Quan Pan. DOEPatch: Dynamically optimized ensemble model for adversarial patches generation. *arXiv preprint arXiv:2312.16907*, 2023.

[41] Simen Thys, Wiebe Van Ranst, and Toon Goedemé. Fooling automated surveillance cameras: Adversarial patches to attack person detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2019.

[42] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. Yolov10: Real-time end-to-end object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[43] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7464–7475, 2023.

[44] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. You only learn one representation: Unified network for multiple tasks. *arXiv preprint arXiv:2105.04206*, 2021.

[45] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv:2402.13616*, 2024.

[46] Donghua Wang, Tingsong Jiang, Jialiang Sun, Weien Zhou, Xiaoya Zhang, Zhiqiang Gong, Wen Yao, and Xiaoqian Chen. FCA: Learning a 3d full-coverage vehicle camouflage for multi-view physical adversarial attack. In *AAAI Conference on Artificial Intelligence*, 2022.

[47] Zhixiang Wang, Xingjun Ma, and Yu-Gang Jiang. Badpatch: Diffusion-based generation of physical adversarial patches. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2025. arXiv:2412.01440.

[48] Kaidi Xu, Gaoyuan Zhang, Sijia Liu, Quanfu Fan, Mengshu Sun, Hongge Chen, Pin-Yu Chen, Yanzhi Wang, and Xue Lin. Adversarial t-shirt! evading person detectors in a physical world. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 665–681, 2020.

[49] Wei Zhang, Zhanhao Hu, Xiao Li, Xiaopei Zhu, and Xiaolin Hu. A single set of adversarial clothes breaks multiple defense methods in the physical world. *arXiv preprint arXiv:2510.17322*, 2025.

[50] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. Detrs beat yolos on real-time object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[51] Zheng Zhou, Hongbo Zhao, Ju Liu, Qiaosheng Zhang, Liwei Geng, Shuchang Lyu, and Wenquan Feng. MVPatch: More vivid patch for adversarial camouflaged attacks on object detectors in the physical world. *arXiv preprint arXiv:2312.17431*, 2024.

7. Research Ethics

This paper evaluates published attacks and proposes no new one. Every method runs from the code its own authors released, so we hand an attacker no capability they did not already have, and our findings lower the threat this literature reports, which helps a defender size an exposure. Every detector is an open-source model that we run locally, nothing is sent to a live service, and the evaluation is entirely digital, so no patch was printed or worn. We use the public INRIA Person benchmark [9], collect no new data, involve no human participants, identify no one in the images, and redistribute none of them.

Appendix

A.1 Open Science Policy

We plan to release the full artifact behind this paper so that any cell of Table 1 can be reproduced on its own. The release will contain each of the 160 patches we trained, as a PNG image for inspection and reprinting and as a PyTorch tensor that preserves the exact pre-quantization val-

© 2026 Information Processing Society of Japan

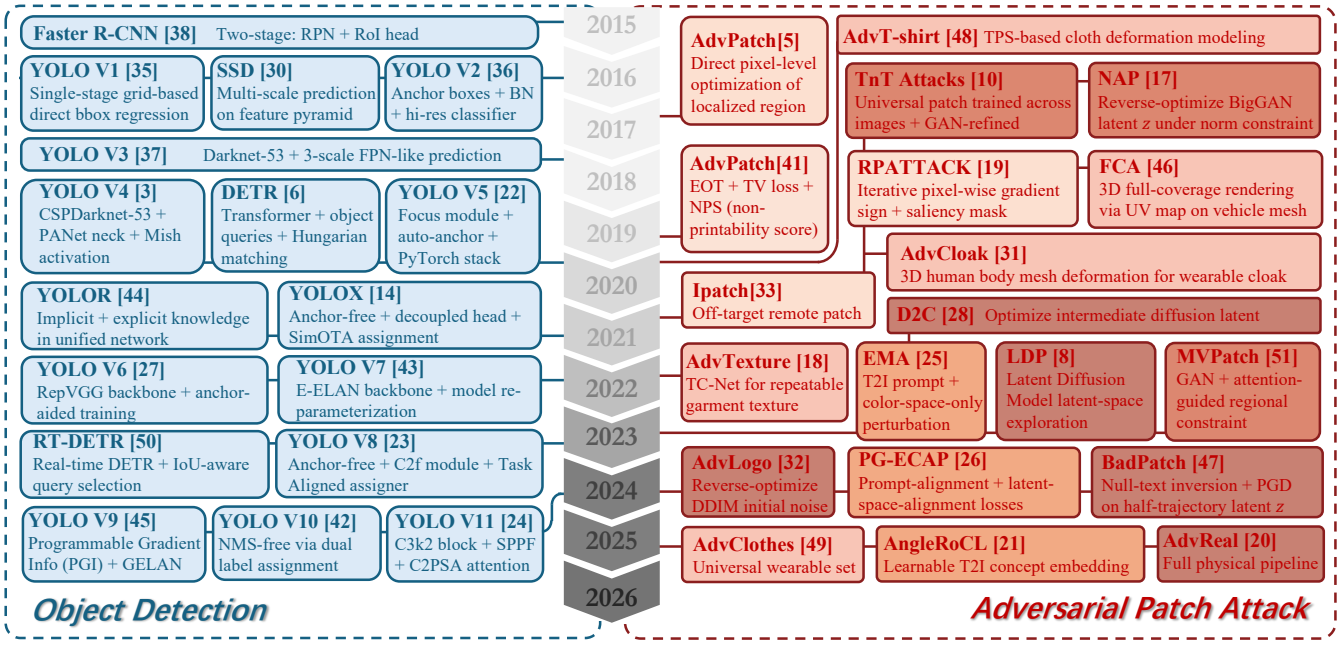


Fig. 6: Object detectors and adversarial patch attacks on one time axis, with the mechanism each work introduced. Detectors run down the left, attacks down the right, and the year sits between them.

ues. It will also carry the configuration and the random seed of every run, the per-detector measurements behind every number we report, and the interface code described in Appendix A.4, which is the only part of any attack pipeline we wrote ourselves, so that a reader can see what we adapted and what we left alone. The INRIA Person dataset and the pretrained detector weights we pin instead of redistributing, since both are public at their original sources.

A.2 Timeline of Detectors and Patch Attacks

Fig. 6 sets the two lines of work on one time axis, the object detectors on the left and the patch attacks on the right, each entry labelled with the mechanism it introduced. It is the long form of Section 2. The detector column runs from the two-stage proposal pipelines through the darknet generations, the anchor-free redesigns and the transformer heads, and the attack column runs from raw pixel optimization through physical robustness and on to generation-model latents and text prompts. Neither side pauses, which is what makes the victims recorded in Section 3 a choice and not a constraint, since for almost every attack in the right column a newer detector already stood in the left one by the time the attack appeared.

A.3 Surveyed Methods by Stage

Section 2.2 sorts the literature by the variable each attack optimizes. Table A.1 lists the 24 methods behind that division one by one, with the core contribution of each, the class it attacks, the victim detectors it reports, and whether we found official code for person evasion. The table carries the two observations of Section 3.1 at the level of individual papers. Faster R-CNN takes a mark in 14 of the 24 rows, so a detector released in 2015 is the most common victim in a list that runs to 2025, while the columns from YOLOv6 onward carry six marks between them and none at all for YOLOv6 or YOLOv9. The table also shows why Section 4.1 can seat only four of the five stages. Twelve of the 24 methods release official code, and the one release in Stage 5 attacks stop signs rather than people, so EMA, PG-ECAP and AngleRoCL together leave that stage with nothing we could run.

A.4 Implementation Details

Section 4.1 states the protocol in the space the main text allows. This appendix gives the parts a reader would need in order to repeat it.

Where an attack's own code already supports a victim detector we run it unchanged. Where it does not, we add the interface that attaches the attack to that detector and leave the rest of the pipeline alone. Two adaptations are forced on us by the detectors themselves. The first is the confidence score that enters the loss: the darknet-style heads expose an objectness branch and the anchor-free heads do not, so we read per-box person confidence as objectness times class probability on the former and as the bare class score on the latter, which is the closest match to what each head actually publishes. The second is how the per-box scores are aggregated into one number. A method takes either the maximum over boxes or the mean over the surviving cluster, and we keep whichever its own authors chose instead of imposing one. No hyperparameter changes when an attack moves to a new detector, and every patch is trained to convergence with the method's own optimizer. Where a

© 2026 Information Processing Society of Japan

Table A.1: The five stages of adversarial patch attacks. • marks a reported victim detector, ✓ and ✗ whether official open-source code exists, and ★ the method we evaluate for that stage. FR is Faster R-CNN, TR a transformer detector, and Ot any other convolutional detector. The shaded rows are Stage 5, which contributes no method to the evaluation.

Stage	Method	Year	Core contribution	Target	YOLO Family										Other			Code	Ours
					2	3	4	5	6	7	8	9	10	11	FR	TR	Ot		
1. Pixel	AdvPatch [5]	2017	Universal patch under transformation	*Classifier*														✓	✗
	AdvPatch [41]	2019	EOT, TV loss and non-printability score	Person	•													✓	★
	RPAttack [19]	2021	Pixel-wise gradient sign, saliency mask	Generic			•								•			✓	✗
	IPatch [33]	2021	Off-target remote patch	*Segmentation*														✗	✗
	Eigenpatches [2]	2023	Linear combination of patch PCA bases	Person						•								✓	✗
2. Physical	AdvT-shirt [48]	2020	TPS-based cloth deformation	Person	•										•			✗	✗
	AdvCloak [31]	2021	3D body-mesh deformation for a cloak	Person	•										•			✗	✗
	FCA [46]	2022	Full-coverage UV rendering on a mesh	*Vehicle*		•		•							•		•	✓	✗
	AdvTexture [18]	2022	TC-Net for repeatable garment texture	Person	•	•									•		•	✓	★
	DAP [16]	2023	Similarity loss and a creases transform	Person	•	•	•			•					•			✗	✗
	DOEPatch [40]	2023	Energy view, dynamic ensemble weights	Person	•	•	•								•			✗	✗
	AdvReal [20]	2025	Joint 2D and 3D texture optimization	Person	•	•		•			•			•	•			✓	✗
	AdvClothes [49]	2025	One wearable set against many defenses	Person											•		•	✓	✗
3. GAN	NAP [17]	2021	BigGAN latent under a norm constraint	Person	•	•	•								•			✓	★
	TnT [10]	2022	Universal patch refined by a GAN	*Classifier*														✗	✗
	GANPatch [34]	2022	Pretrained GAN against GAN co-training	Generic		•												✗	✗
	MVPatch [51]	2024	GAN with attention-guided region loss	Person	•	•	•								•			✗	✗
4. Diffusion	D2C [28]	2023	Optimize the intermediate latent	Person	•	•	•								•			✗	✗
	LDP [8]	2023	Latent-space exploration by denoising	Person														✗	✗
	AdvLogo [32]	2024	Reverse-optimize the DDIM initial noise	Person	•	•	•	•							•		•	✓	✗
	BadPatch [47]	2025	Null-text inversion, PGD on a half latent	Person		•	•	•		•								✓	★
5. Prompt	EMA [25]	2024	T2I prompt, colour-space perturbation	*Aerial*				•										✗	✗
	PG-ECAP [26]	2024	Prompt and latent alignment losses	*Scene*														✗	✗
	AngleRoCL [21]	2025	Learnable angle-robust concept embedding	*Stop sign*		•		•					•		•	•		✓	✗

method's pipeline already models a physical factor, such as the cloth deformation AdvTexture trains against, we keep that machinery as part of reproducing the method faithfully.

Every patch is rendered onto the person at one common scale of 0.2, and for a patch that is not a square we match the occupied area to that same footprint, so a small square patch is never scored against a full-garment texture. Each patch keeps its own method's official texture resolution. That choice does not change the area a patch occupies, because every patch is resized to the footprint before it is composited.

Every cell of Table 1 is a single training run. We hold the random seed fixed across the whole evaluation, and the two attacks that condition on a class keep one class throughout, NAP the peacock class of BigGAN and BadPatch the prompt "anime, a colorful cat sitting on the ground with its eyes open, cute, cat, hd wallpaper". Neither method treats that choice as something to optimize and neither paper reports it, so we record ours here and hold it fixed so that every cell stays a configuration one attacker could reproduce.

We do not claim a bit-exact reproduction of any original paper, since seeds and library versions differ from theirs, so our absolute numbers are not meant to be read against the ones those papers print. What we claim is internal consistency. Every attack runs from its own official code with its own hyperparameters, nothing is retuned when it moves to a new detector, and every cell is scored under the same protocol, so a comparison drawn inside Table 1 compares attacks and detectors and not setups.

© 2026 Information Processing Society of Japan